

让 NPU 更易共享： HAMi vNPU 技术演进与招行落地实践

王纪飞（密瓜智能）

社区协作 · 技术演进 · 联合验证

目录

- 01 第一部分 | 社区与招行合作的渊源
- 02 第二部分 | vNPU 的演进 · 硬切分
- 03 第二部分 | vNPU 的演进 · 软切分尝试
- 04 第二部分 | vNPU 的演进 · 软切分实现
- 05 第二部分 | vNPU 的演进 · 软切分应用
- 06 第二部分 | vNPU 的演进 · 未来展望

第一部分：社区与招行的合作

长期联系，让需求、技术与验证形成连续闭环

去年

项目协作

联合验证

建立联系

通过社区驱动持续沟通，形成与招商银行长期协作的基础。

走进场景

参与招行沐曦、昆仑芯等项目，理解异构算力的真实需求。

共同推进

围绕昇腾共享能力，将社区实现带入真实环境验证。

招商银行既是使用者，也是社区迭代的重要输入



需求输入

提供异构算力场景中的真实约束、优先级与使用体验。



方向校准

帮助社区判断哪些能力真正影响落地，避免闭门造车。



资源验证

提供设备与环境，验证实现的兼容性、稳定性与可用性。

第二部分：vNPU 的演进

硬切分 | 能用，但离灵活共享还有距离

不同型号支持不同的固定切分类型，模板由厂商预先定义。资源分配看似精确，实际上仍受硬编码边界约束。



规格不统一

有的设备一切四，有的一切七；
跨型号体验难一致。



模板硬编码

只能选择预设组合，业务无法按
实际负载弹性调整。



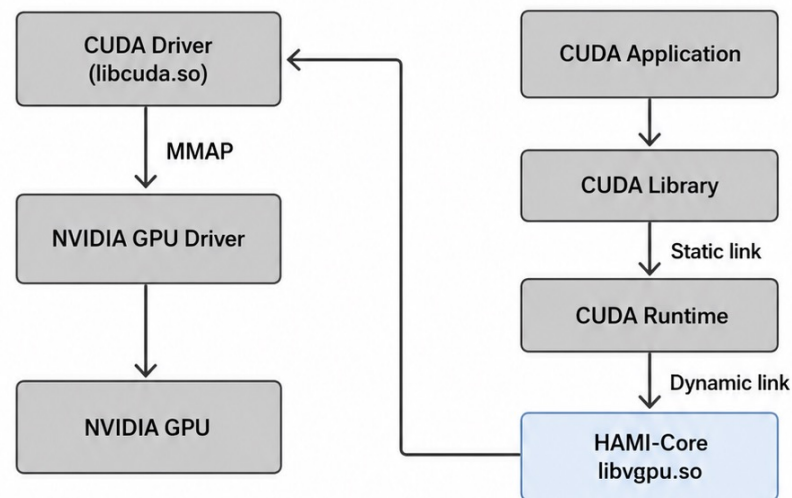
资源强绑定

算力限制与显存限制绑定，只按
显存对齐，利用率受限。

硬切分 | 软切分受阻于底层 API

GPU 路径

关键运行时 **API** 可被拦截与重定向，社区能够在进程与设备之间实施资源控制。

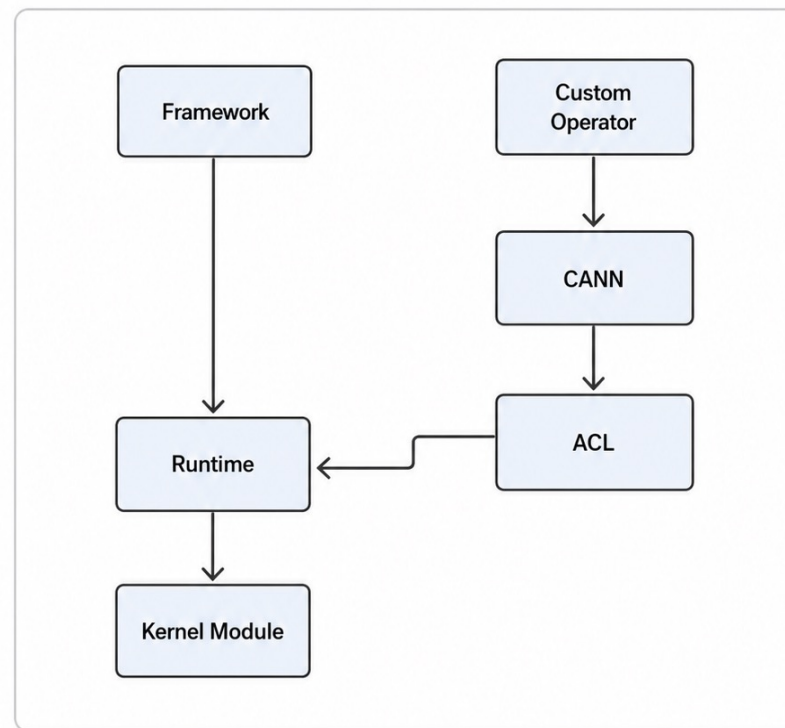


NPU 路径

当时关键底层 **API** 没有开放，无法完成同等程度的劫持与控制。

结论

没有可控的执行入口，上层 **Device Plugin** 只能分配设备，无法实现真正的细粒度共享。



软切分尝试 | 技术条件经历了三次关键变化

适配窗口出现

底层软件栈开放后，社区开始准备支持昇腾共享。

发现多进程限制

实际实验表明，设备当时不支持期望的多进程并发调用。

共享基础成熟

驱动升级支持多进程，核心库能力成为可行路径。

● CANN 开源

● 实验验证

● 驱动升级

软切分尝试 | Flex:AI 展示了另一条绕行路径

华为开源 Flex:AI，目标是共享昇腾 NPU；但其实现仍遵循当时的设备限制：通过远程调用汇聚请求，NPU 侧实际上保持单进程执行。

表面：多个任务共享 NPU

- 远程转发请求
- 共享发生在调用入口
- 隔离依赖服务层

设备侧：仍由一个进程执行

- 遵循多进程限制
- 不是本地多进程并发
- 部署链路更复杂

软切分实现 | 多进程支持让共享可直接控制

驱动能力

核心库

工程入口

设备可并发

昇腾驱动升级后支持多进程，为细粒度共享提供底座。

hami-core 类似能力

华为加拿大实验室据此开发底层共享控制库。

进入 Kubernetes

底层控制能力可以被 Device Plugin 与调度体系承接。

软切分实现 | Kubernetes 落地生产

1. 发现与注册

Device Plugin 识别昇腾设备并向 Kubernetes 暴露可调度资源。

2. 分配与注入

调度结果转化为容器可见的设备与共享配置。

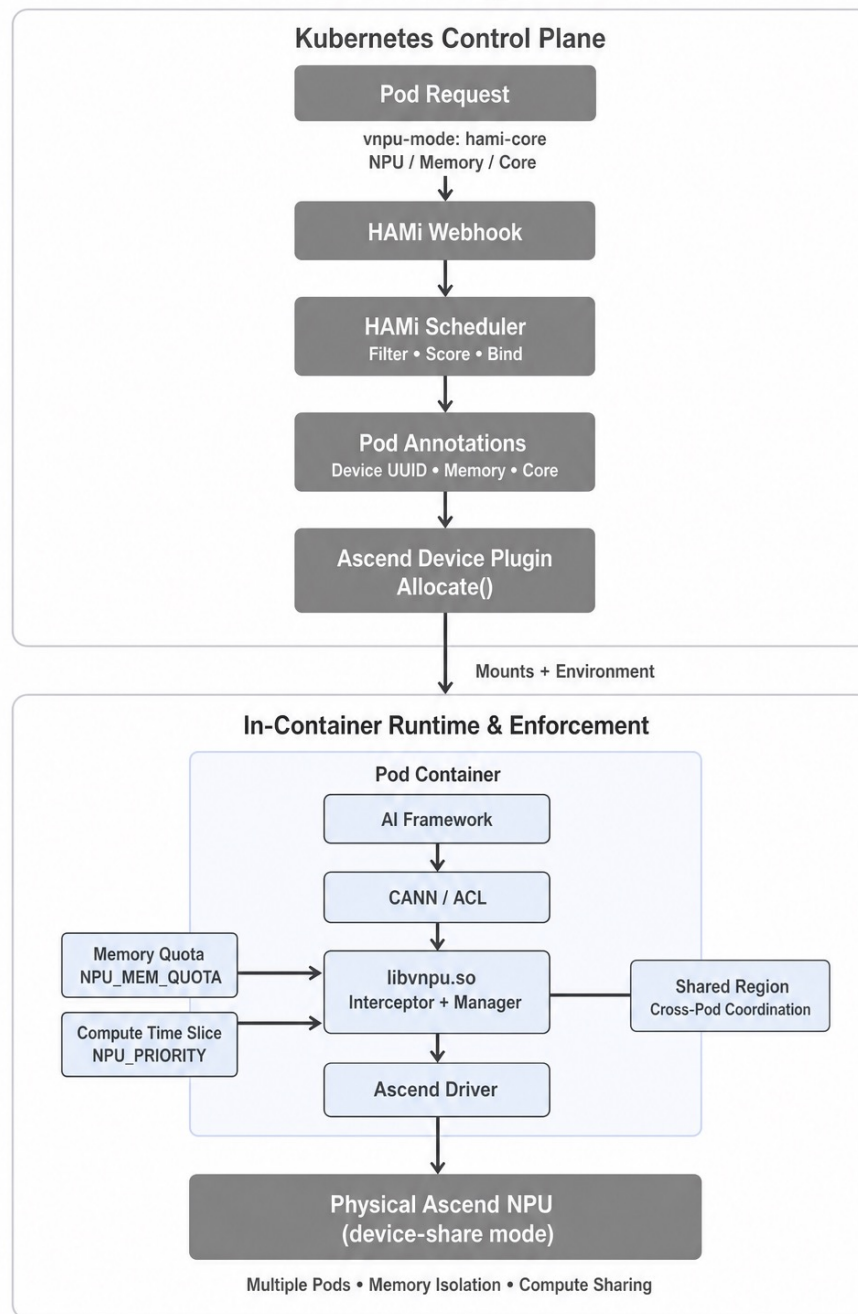
3. 运行时控制

底层库在多进程条件下实施资源共享与限制。

4. 闭环验证

把可用性、稳定性和体验问题带回社区迭代。

HAMi vNPU Soft Slicing Architecture



软切分应用 | 从实验室走向生产

HAMi 社区反馈：生产落地时用户要处理复杂的配置，**API** 的不一致，独特的行为表现，心智负担很重，需要不断优化。

切分结果容器内不可见

未劫持 **dsmi**，容器内执行 **npu-smi info** 仍看到整卡信息。

驱动配置

要求特定驱动版本，并手动配置启用 **npu-share** 能力。

算力分配逻辑不一致

请求超过剩余资源，应稳定 **Pending** 并给出原因，不能让调度器异常退出。

切片分配保留时限

新分配的 **vnpu** 设备在一段时间内没有被使用会自动回收。

软切分应用 | 2 / 4 / 8 / 16 切分的性能拐点

测试环境：910C单卡、7B 级模型、输入 200 / 输出 128 token、每个 vNPU 单请求、P50

切分规格	TTFT（ms）	TPOT（ms/token）	单卡实例数	场景判断
1 / 2	78	24.0	2	强时延 SLA
1 / 4	92	29.7	4	线上推荐平衡点
1 / 8	134	43.9	8	密度优先 / 宽松 SLA
1 / 16	248	76.4	16	开发测试 / 离线
评价方向	越低越好	越低越好	越高密度越高	满足 SLA 的最细档
测试口径	同模型同请求	同输入输出长度	单卡并发	补充 batch / 并发度
关键观察	关注增幅拐点	关注持续生成抖动	密度换时延	不是越细越好
数据说明	P50 估算	P50 估算	每 vNPU 1 实例	

软切分应用 | 四切是在线服务的推荐平衡点

四切相对二切只增加 18% TTFT、24% TPOT；八切与十六切则用更明显的时延代价换取实例密度。

1 / 2 | 时延优先

TTFT 78 ms | TPOT 24.0 ms

适合交互强、首 token 和生成速度要求高的服务。

1 / 8 | 密度换时延

TTFT 134 ms | TPOT 43.9 ms

适合轻量模型、异步请求或更宽松的 SLA。

1 / 4 | 推荐平衡点

TTFT 92 ms | TPOT 29.7 ms

时延仍可控，同时把单卡实例密度提升到 4。

1 / 16 | 非在线首选

TTFT 248 ms | TPOT 76.4 ms

适合开发测试、低频服务和离线任务。

未来展望 | 体验改善：让共享真正可用、可管

先补齐现有实现的产品化能力：用户少记规则，平台看得清状态，升级有边界，异常能恢复。

用户 API 一致化

硬切分软切分统一管理，任务按需分配，对上层无感知。

兼容矩阵强化

同步更新 volcano 昇腾 device-plugin。

可观测性补齐

支持容器内切分结果可见，补齐容器级别算力显存使用监控。

使用体验优化

优化 values 配置，简化配置格式，常用配置自动化修改，补齐报错日志，常见问题可以用户自主排查。

未来展望 | 进阶功能：从静态切分走向动态治理

当资源边界可控、状态可见之后，下一阶段是理解负载、动态调整并提高整体利用率。



Profiling | 看懂负载

采集算子、显存、算力与时延特征，识别瓶颈和任务间干扰，为调度提供依据。



算力抢占 | 保障优先级

高优先级任务到来时，动态回收或压缩低优先级算力，同时控制恢复成本与尾时延。



资源超卖 | 释放闲置

基于真实使用而非静态申请共享余量；配合风险水位、限流和回退机制避免失控。

感谢聆听



HAMi 社区小助手



密瓜智能官方网站



HAMi 官方公众号

GitHub:

<https://github.com/project-hami/hami>